

ELE3021_Project #1 Wiki

컴퓨터소프트웨어학부 2020003309 최성민

Design

1. MLFQ 구현

MLFQ (Multi-Level Feedback Queue) 알고리즘을 구현합니다.

필요한 구조체는 다음과 같습니다.

- Node : 큐에 들어갈 프로세스 정보 저장
- Node Memory Pool : Allocation을 줄이기 위한 역할
- Queue : Node Pointer와 Count 저장
- MLFQ : 3개의 Level을 구분짓는 Queue를 저장하고 있는 구조체

proc.c 파일에서 호출할 다음 함수들이 준비되어야 합니다.

- mlfq init : MLFQ 구조체를 초기화합니다.
- mlfq push_front : 주어진 데이터가 MLFQ의 제일 첫 순서가 되도록 PUSH합니다.
- mlfq push : 주어진 데이터를 MLFQ에 PUSH합니다.
- mlfq pop : MLFQ에서 POP합니다.
- mlfq pop_target : 주어진 타겟 데이터를 큐에서 빼냅니다.

L0 Queue와 L1 Queue는 RR (Round-Robin) 정책을 따르기 때문에, FCFS로 동작하는 Queue를 이용해도 문제가 없습니다.

L2 Queue는 Priority Scheduling 정책을 따르며, Priority가 같은 Process는 FCFS로 동작합니다. Priority의 범위는 0 ~ 3입니다. 따라서 4개의 Queue를 이용하여 Process를 Priority에 맞는 Queue에 삽입한 후 관리합니다.

2. MLFQ 알고리즘 Scheduler 구현

MLFQ를 이용하는 Scheduler는 간단하게 구현됩니다.

MLFQ는 Ready Queue의 역할로 활용되며, Runnable한 Process만 push됩니다.

Process의 상태를 RUNNABLE로 바꾸는 부분 아래에 Process를 MLFQ로 삽입하는 로직을 추가합니다.

Scheduler에서 MLFQ_POP을 진행한 후 해당 프로세스를 스케줄링합니다.

3. Time Quantum 적용

현재 xv6는 1tick마다 Process를 yield 시킵니다.

piazza를 통해 알아본 결과, Process는 1tick마다 CPU를 양보하되, Level과 Priority는 주어진 Time Quantum마다 변경되어야 함을 이해했습니다. 또한 CPU에 Process가 올라감과 동시에 Time Quantum은 1이 올라가야 합니다.

따라서 Scheduler에서 Process를 선택하여 CPU에 올리는 순간 Process의 tick을 올려 줍니다. 또한 1tick마다 Process가 yield되므로, Process가 yield된 후 MLFQ에 Process가 삽입될 때, Level과 Priority를 조정해줍니다.

4. Priority Boosting 적용

현재 global tick인 trap.c의 ticks는 sleep 함수 등 여러 곳에서 사용될 여지가 있습니다. 따라서 매 100 tick마다 0으로 초기화해주는 것은 힘들 것으로 보입니다. 새로운 boosting_ticks를 추가해줍니다. 이 boosting_ticks는 기존 global tick인 ticks와 함께 증감해야 하므로, ticks의 lock인 tickslock을 그대로 이용해줍니다.

piazza를 통해 알아본 결과, Priority Boosting은 현재 동작하는 Process가 없어도 일어나야 합니다. 따라서 trap.c에 myproc이 0일 경우에 Priority Boosting을 하는 로직이 필요합니다.

Boosting을 하는 경우, CPU에 Process가 올라가 있으면 안됩니다. 따라서 myproc이 존재하는 경우에 공통적으로 실행되는 sched 함수의 처음 부분에 Priority Boosting을 하는 로직을 추가합니다.

5. System Call 구현

yield, getLevel은 구현하기 쉽습니다.

MLFQ는 Process의 Level과 Priority를 참고하여 L0, L1, L2-0, L2-1, L2-2, L2-3 Queue 중 어디에 Process를 삽입할 지 결정합니다. setPriority를 통해 해당 Priority를 바꿀 경우, 이미 삽입된 Process를 한번 뺀 후 다시 삽입해야 합니다.

schedulerLock과 schedulerUnlock은 프로세스의 스케줄링 우선순위를 최우선으로 설정합니다.

- schedulerLock을 Process A가 부르고 난 후 Lock을 유지한 상태에서 B가 불렀을 경우, B는 Lock을 받을 수 있을 때까지 기다립니다. A의 Lock이 풀릴 경우 B는 Lock을 이어받습니다. Process B는 Lock을 호출하며 Lock과 Unlock 사이의 코드가 Atomic하게 실행되길 바랄겁니다. 따라서 Lock을 받을 수 있을 때까지 기다립니다.
- schedulerLock을 Process A가 부르고 난 후 Lock을 유지한 상태에서 A가 다시 불렀을 경우, boosting_ticks를 0으로 초기화합니다. Lock을 100ticks보다 길게 받으려고 하는 경우를 핸들링하기 위함입니다.
- schedulerUnlock을 Process A가 부르고 난 후 B가 불렀을 경우 Lock이 되어있지 않는 상황에서 B가 Unlock을 호출했습니다. 이 상황에서는 아무 행동도 하지 않고 return을 시킵니다. schedulerUnlock을 Process A가 부르고 난 후 A가 다시 불렀을 경우도 동일합니다.

schedulerUnlock이 성공적으로 실행되었을 경우와 Priority Boosting을 통해 Lock이 풀린 경우, 현재 Lock을 대기하는 Process가 있으면 Process를 깨웁니다. 이때 boosting_ticks가 1000이라면 Priority Boosting을 진행합니다.

6. Interrupt 추가

schedulerLock과 schedulerUnlock은 Interrupt를 통해 호출되어야 합니다. 128번, 129번 Interrupt를 활용하고 trap.c에서 받도록 합니다.

Implement

1. MLFQ 구현입니다.

Ln : Level이 n인 Process가 모인 Queue

Pn : Priority가 n인 Process가 모인 Queue

L0 : mlfq->queues[0]

L1 : mlfq->queues[1]

L2 : mlfq->pri_queues

P0 : mlfq->pri_queues[0]

P1 : mlfq->pri_queues[1]

P2 : mlfq->pri_queues[2]

P3 : mlfq->pri_queues[3]

```
typedef struct node {
    struct proc* proc; //
    struct node* prev;
    struct node* next;
} node;

typedef struct queue {
    node* front;
    node* rear;
    int count;
} queue;

typedef struct mlfq {
    queue* queues[2]; //
    queue* pri_queues[4];
} mlfq;
```

mlfq* mlfq_init();

node memory pool을 초기화하고, mlfq를 초기화합니다.

void mlfq_push_front(mlfq* q, struct proc* p);

L0 Queue front에 해당 Process를 삽입합니다.

SchedulerLock이 풀릴 때 사용합니다.

void mlfq_push(mlfq* q, struct proc* p);

Process의 time quantum을 확인 후, Level과 Priority를 업데이트합니다.

그 후 Level과 Priority에 맞게 Process를 MLFQ에 삽입합니다.

struct proc* mlfq_pop(mlfq* q);

우선 순위가 가장 먼저인 Process를 빼냅니다.

Process가 없다면 0을 반환합니다.

struct proc* mlfq_pop_target(mlfq* q, struct proc* p);

두번째 Arg로 주어진 Process를 MLFQ에서 빼냅니다.

Process가 없다면 0을 반환합니다.

```
void mlfq_boost(mlfq* q);
```

MLFQ에 들어있는 RUNNABLE한 Process들의 Level과 Priority를 초기화합니다.
따라서 모든 Queue의 Process를 L0 Queue로 이동시킵니다.
Priority Boosting에서 사용됩니다.

위의 함수를 구현하기 위해 Queue와 Node를 조정하기 위한 추가적인 함수들이 구현되었습니다.

2. Scheduler 구현입니다.

Process 구조체인 proc에는 Scheduling을 위한 level과 priority, time_quantum 변수가 추가되었습니다. 또한 SchedulerLock 구현에서 쓸 next 변수도 추가되었습니다.

```
struct {  
    struct spinlock lock;  
    struct proc proc[NPROC]; // Num of Process  
    struct mlfq* mlfq; // multi level feedback queue  
    struct proc* lockproc;  
} ptable;
```

Process Data가 모두 저장되는 ptable 구조체입니다. Ready Queue로 사용될 MLFQ와 SchedulerLock에 사용될 lockproc 변수가 추가되었습니다.

```
found:  
    p->state = EMBRYO;  
    p->pid = nextpid++;  
    p->level = 0;  
    p->priority = 3;  
    p->time_quantum = 4;  
    p->next = 0;
```

allocproc 함수에서 할당될 Process를 초기화해주므로, 추가한 변수를 초기화해줍니다.

```
p->state = RUNNABLE;  
mlfq_push(ptable.mlfq, p);
```

userinit과 fork, wakeup1, kill 함수에서는 Process의 state를 RUNNABLE로 바꿉니다.
이때 state가 바뀌는 Process는 Ready Queue에 들어가야 하므로, 아래에 mlfq_push 함수를 추가하여 해당 케이스를 핸들링합니다.

Scheduler 함수를 다음과 같이 바꿉니다.

```
void
scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();
    c->proc = 0;

    for(;;){
        sti();

        acquire(&ptable.lock);

        if (ptable.lockproc && ptable.lockproc->state == RUNNABLE)
            scheduler1(c, ptable.lockproc);
        else if((p = mlfq_pop(ptable.mlfq)) && p->state == RUNNABLE)
            scheduler1(c, p);

        release(&ptable.lock);
    }
}
```

lockproc은 SchedulerLock에서 설명드리겠습니다.

mlfq_pop을 통해 Scheduling 후보 Process를 가져옵니다. 해당 Process가 RUNNABLE이 아닌 경우는 루프를 거쳐 다시 후보를 뽑아옵니다. RUNNABLE인지 확인하는 이유는, Ready Queue에 들어있더라도 Process 동작 과정에서 모종의 이유로 상태가 바뀔 수 있기 때문입니다.

```
void
scheduler1(struct cpu *c, struct proc *p)
{
    c->proc = p;
    p->time_quantum--;

    switchvm(p);
    p->state = RUNNING;
    swtch(&(c->scheduler), p->context);
    switchkvm();
    c->proc = 0;
}
```

가독성과 코드 재활용을 위해, 선택된 Process를 CPU에 올리는 과정을 scheduler1 함수로 뺍니다. Process는 CPU에 올라가자마자 time quantum에 반영됩니다. 아래 Tick 부분에서 동작 과정을 설명드리겠습니다.

3. Time Quantum 적용과 Priority Boosting입니다.

```
struct spinlock tickslock;  
uint ticks;  
uint boosting_ticks;
```

trap.c 의 ticks 아래에 boosting_ticks를 추가해줍니다.

```
case T_IRQ0 + IRQ_TIMER:  
    if(cpuid() == 0){  
        acquire(&tickslock);  
        ticks++;  
        wakeup(&ticks);  
  
        if (boosting_ticks >= 100){  
            if (myproc() && myproc()->state == RUNNING){  
                // sched 문에서 처리  
            }  
            else{  
                boosting();  
                boosting_ticks = 0;  
            }  
        }  
        else boosting_ticks++;  
        release(&tickslock);  
    }  
}
```

TIMER Interrupt가 호출될 때, ticks를 증가하는 로직에 boosting_ticks를 증가하는 로직을 추가해줍니다. 이때, boosting_ticks가 100보다 클 경우, Priority Boosting을 진행해야 하므로 조건문을 추가해줍니다.

이때, 현재 RUNNING중인 Process가 존재하는 경우, sched 함수에서 boosting을 처리하도록 로직을 구현했으므로, 해당 경우가 아닐 때만 Priority Boosting을 수행하는 boosting 함수를 호출해줍니다.

```
void  
boosting(void)  
{  
    acquire(&ptable.lock);  
  
    if (ptable.lockproc)  
    {  
        if (ptable.lockproc->next) wakeup1(ptable.lockproc->next);  
        mlfq_push_front(ptable.mlfq, ptable.lockproc);  
    }  
  
    mlfq_boost(ptable.mlfq);  
    release(&ptable.lock);  
}
```

lockproc에 대해서는 아래 SchedulerLock에서 설명드리겠습니다.

MLFQ 구현에서 설명드린 mlfq_boost 함수를 호출합니다.

```
void
sched(void)
{
    int intena;
    struct proc *p = myproc();

    // 현재 프로세스가 없을 때 부스팅을 진행해야 순서가 안 꼬임.
    if(cpuid() == 0){
        acquire(&tickslock);
        if (boosting_ticks >= 100)
        {
            mlfq_boost(ptable.mlfq);
            boosting_ticks = 0;
        }
        release(&tickslock);
    }
}
```

sched 함수에도 mlfq_boost 함수를 호출합니다. boosting_ticks를 0으로 초기화해주므로 tickslock을 이용합니다.

```
void
mlfq_boost(mlfq* q)
{
    for (int i = 0; i < 2; i++)
        for (node *c = q->queues[i]->front; c != 0; c = c->next) {
            c->proc->level = 0;
            c->proc->priority = 3;
            c->proc->time_quantum = 4;
        }

    for (int i = 0; i < 4; i++)
        for (node *c = q->pri_queues[i]->front; c != 0; c = c->next) {
            c->proc->level = 0;
            c->proc->priority = 3;
            c->proc->time_quantum = 4;
        }

    // from L1 to L0
    while (!queue_is_empty(q->queues[1])) {
        queue_push(q->queues[0], queue_pop(q->queues[1]));
    }

    // from L2 to L0
    // 최근에 실행된 애들은 더 뒤에 있음. 실행 안된 애들이 제일 앞. FCFS를 지키기 위해서 Priority 3부터 부스
    for (int i = 3; i >= 0; i--)
        while (!queue_is_empty(q->pri_queues[i])) {
            queue_push(q->queues[0], queue_pop(q->pri_queues[i]));
        }

    // cprintf("L0 %d L1 %d L2 %d -> L0 %d\n",
    //         count[0], count[1], count[2], count[0] + count[1] + count[2]);
}
```


mlfq_boost 함수는 위와 같이 구현되어 있습니다. 모든 Queue의 Process의 Level, Priority, Time Quantum을 기본값으로 초기화해줍니다. 그리고 L0 Queue에 넣어줍니다.

Priority Queue의 경우, 순서가 보존되어야 합니다. 이때, Priority가 3인 Queue부터 2, 1, 0 순으로 FCFS 정책을 적용하여 L0 Queue에 삽입을 하면, 순서가 보장됩니다.

4. System Call 구현

yield 함수의 경우 이미 System Call로 구현되어 있습니다.

```
int
getLevel(void)
{
    return myproc()->level;
}
```

getLevel 함수의 경우 위와 같이 구현했습니다.

```
void
setPriority(int pid, int priority)
{
    if (3 < priority || priority < 0) return;

    struct proc *p = 0;

    acquire(&ptable.lock);
    for(p = ptable.proc; p < &ptable.proc[NPROC]; p++)
        if (p->pid == pid) break;
    if (!p || p->priority == priority)
    {
        release(&ptable.lock);
        return;
    }

    if (p->state == RUNNABLE && p->level == 2)
    {
        p = mlfq_pop_target(ptable.mlfq, p);
        if (!p)
        {
            release(&ptable.lock);
            return;
        }

        int ori_priority = p->priority;
        p->priority = priority;
        if (p->state == RUNNABLE && p->level == 2)
        {
            if (ori_priority > p->priority) mlfq_push_front(ptable.mlfq, p, 1);
            else mlfq_push(ptable.mlfq, p);
        }

        release(&ptable.lock);
    }
}
```

setPriority 함수는 위와 같이 구현되었습니다.

입력된 Priority를 확인합니다. 범위 밖일 경우 동작하지 않고 Return합니다.

Process가 존재하지 않을 경우, priority를 바꿀 필요가 없는 경우 Return합니다.

Priority를 조정하기 전, 현재 Process가 들어있는 Queue의 위치를 조절해줄 필요가 있다면, 먼저 해당 Process를 찾아 Ready Queue에서 빼줍니다. 그리고 Priority를 조정한 다음 다시 Ready Queue에 삽입해주어 추후 Scheduling 후보 Process 선택 과정에서 차질이 없도록 합니다.

이때, Priority가 갱신되는 경우 FCFS가 유지되어야 하므로, Priority가 작아진 경우, 순서상 앞에 위치해 있었으므로 먼저 실행되게 Front에 삽입하며 Priority가 커진 경우 순서상 뒤에 위치해 있었으므로 Back에 삽입합니다.

5. System Call – SchedulerLock Unlock 구현

```
struct {
    struct spinlock lock;
    struct proc proc[NPROC]; // Num of Process
    struct mlfq* mlfq; // multi level feedback queue
    struct proc* lockproc;
} ptable;
```

lockproc을 추가했습니다. 해당 변수는 0일 때, Unlock 상태이며, Process가 할당되어 있을 경우 Lock 상태입니다.

```
void
schedulerLock(int password)
{
    if (!check_pw(password)) return;
    acquire(&ptable.lock);

    if (ptable.lockproc && ptable.lockproc != myproc()) {
        struct proc* p = ptable.lockproc;
        while(p->next) p = p->next;
        p->next = myproc();
        sleep(myproc(), &ptable.lock);
    }

    acquire(&tickslock);
    ptable.lockproc = myproc();
    boosting_ticks = 0;
    release(&tickslock);

    release(&ptable.lock);
}
```

schedulerLock 함수입니다. Password를 먼저 체크한 후, 현재 Scheduler가 Lock 상태인지 확인합니다. 이때, Lock이 걸려 있더라도 Lock을 건 Process가 현재 schedulerLock을 호출한 Process와 동일하다면 성공적으로 함수가 실행되었다고 간주합니다.

Lock이 걸려 있지 않은 경우, 현재 Process를 ptable의 lockproc에 할당합니다. 앞으로 이 lockproc은 우선적으로 스케줄링됩니다.

Lock이 걸려있고 Lock을 건 Process가 현재 schedulerLock을 호출한 Process와 달라 성공적으로 함수를 실행하지 못한 경우, 현재 Process는 sleep 상태로 대기합니다. 이때, proc 구조체의 next를 활용하여 lockproc의 next로 이어진 linked list 끝에 매달아둡니다.

```
void
schedulerUnlock(int password)
{
    if (!check_pw(password)) return;
    if (ptable.lockproc && ptable.lockproc == myproc())
    {
        acquire(&ptable.lock);
        ptable.lockproc->level = 0;
        ptable.lockproc->priority = 3;
        ptable.lockproc->time_quantum = 4;
        mlfq_push_front(ptable.mlfq, ptable.lockproc, 0);

        if (ptable.lockproc->next)
        {
            acquire(&tickslock);
            wakeup1(ptable.lockproc->next);
            boosting_ticks = 0;
            release(&tickslock);
        }
        else ptable.lockproc = 0;
        release(&ptable.lock);
    }
}
```

schedulerUnlock 함수입니다. Password를 먼저 체크한 후, 현재 Scheduler가 Lock 상태인지 확인합니다. Lock이 걸려 있으며, Lock을 건 Process가 현재 schedulerLock을 호출한 Process와 동일하다면 성공적으로 함수가 실행되었다고 간주합니다.

Scheduling과 관련된 lockproc의 변수들을 초기화해주고 MLFQ의 순서상 제일 앞쪽에 위치시켜줍니다. 그 후, lockproc의 next에 매달린 Process가 존재한다면, wakeup1을 통해 sleep 상태에서 깨워줍니다. 깡 Process는 schedulerLock 함수를 이어서 실행하므로 다음 lockproc로 할당됩니다.

```

int
check_pw(int pw)
{
    if (pw != SCHED_LOCK_PW)
    {
        struct proc* p = myproc();

        int pid = p->pid, level = p->level;
        int quantum = 2 * p->level + 4 - p->time_quantum;

        cprintf("kill (pid:%d) (time quantum:%d) (level:%d) in schedulerLock for wrong password..
                pid, quantum, level);
        kill(p->pid);
        return 0;
    }
    else return 1;
}

```

Password 체크 함수입니다. 만약 Password가 틀릴 경우, 명세의 요구사항 문구를 출력 해줍니다. 이때, quantum을 따로 연산해주는 이유는 다음과 같습니다. time_quantum으로 Process의 MLFQ 위치를 조정하는 과정에서 $2n+4$ 의 연산이 계속 들어가지 않도록, time_quantum을 증가하는 형식이 아니라 감소하는 형식으로 활용하고 있습니다.

CPU에 할당되는 즉시

3 → 2 → 1 → 0 → 5 → 4 → 3 → 2 → 1 → 0 → 7 ... 로 동작합니다.

따라서 한 차례 연산을 통해 요구사항에 맞게 바꿔줍니다.

```

void
yield(void)
{
    acquire(&ptable.lock); //DOC: yieldlock

    myproc()->state = RUNNABLE;

    if (!ptable.lockproc || boosting_ticks < 100) mlfq_push(ptable.mlfq, myproc());
    else
    {
        if (ptable.lockproc->next) wakeup1(ptable.lockproc->next);
        mlfq_push_front(ptable.mlfq, ptable.lockproc, 0);
        ptable.lockproc = 0;
    }

    sched();
    release(&ptable.lock);
}

```

위는 yield 함수입니다. 만약 boosting 조건을 만족하였고 현재 lockproc이 할당된 경우, Unlock 상태로 초기화해줍니다.

```
void
boosting(void)
{
    acquire(&ptable.lock);

    if (ptable.lockproc)
    {
        if (ptable.lockproc->next) wakeup1(ptable.lockproc->next);
        mlfq_push_front(ptable.mlfq, ptable.lockproc, 0);
    }

    mlfq_boost(ptable.mlfq);
    release(&ptable.lock);
}
```

현재 Process가 동작하고 있지 않은 경우의 boosting 또한 마찬가지입니다.

```
void
scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();
    c->proc = 0;

    for(;;){
        sti();

        acquire(&ptable.lock);

        if (ptable.lockproc && ptable.lockproc->state == RUNNABLE)
            scheduler1(c, ptable.lockproc);
        else if((p = mlfq_pop(ptable.mlfq)) && p->state == RUNNABLE)
            scheduler1(c, p);

        release(&ptable.lock);
    }
}
```

scheduler 함수입니다.

현재 lockproc이 할당되어 있고, 할당된 Process가 RUNNABLE인 경우 CPU에 할당해줍니다. 만약 RUNNABLE하지 않은 경우에는 MLFQ의 후보 Process에게 CPU를 양보해줍니다.

마지막으로, exit 함수에서 lockproc에 할당된 Process 가 종료될 경우, Lock 상태를 풀어줍니다.

```
7 // unlock exit proc
8 if (curproc == ptable.lockproc)
9 {
10     if (ptable.lockproc->next)
11     {
12         acquire(&tickslock);
13         wakeup1(ptable.lockproc->next);
14         boosting_ticks = 0;
15         release(&tickslock);
16     }
17     else ptable.lockproc = 0;
18 }
```

6. Interrupt 호출 처리

```
void
idtinit(void)
{
    lidt(idt, sizeof(idt));

    SETGATE(idt[T_SCHED_LOCK], 1, SEG_KCODE<<3, vectors[T_SCHED_LOCK], DPL_USER);
    SETGATE(idt[T_SCHED_UNLOCK], 1, SEG_KCODE<<3, vectors[T_SCHED_UNLOCK], DPL_USER);
}
```

IDA 테이블에 Interrupt를 추가해줍니다.

```
if(tf->trapno == T_SCHED_LOCK){
    schedulerLock(2020003309);
    return;
}
if(tf->trapno == T_SCHED_UNLOCK){
    schedulerUnlock(2020003309);
    return;
}
```

trap 함수에 해당 조건문을 추가하여 Interrupt 호출을 처리해줍니다.

Result – Scheduling Test

```
int scheduler_test()
{
    parent = getpid();
    int i, pid;
    int count[MAX_LEVEL] = {0};

    printf(1, "[Test 1] Scheduler Test started\n");
    fork_children(4);
    if (getpid() != parent)
    {
        uint tick0 = uptime();

        pid = getpid();
        for (i = 0; i < NUM_LOOP; i++)
        {
            int x = getLevel();
            if (x < 0 || x > 2)
            {
                printf(1, "Wrong level: %d\n", x);
                exit();
            }
            count[x]++;
        }

        schedulerLock(PASSWORD);
        printf(1, "Process %d\n", pid);
        for (i = 0; i < MAX_LEVEL; i++)
            printf(1, "L%d: %d\n", i, count[i]);
        printf(1, "걸린 시간 : %d\n", uptime() - tick0);
        schedulerUnlock(PASSWORD);
    }
    exit_children();
    printf(1, "[Test 1] finished\n");
    printf(1, "done\n");
    return 0;
}
```

테스트 코드입니다.

```
mlfq_test starting on
$ mlfq_test
[Test 1] Scheduler Test started
Process 6
L0: 15822
L1: 23895
L2: 60283
걸린 시간 : 3899
Process 5
L0: 15716
L1: 23729
L2: 60555
걸린 시간 : 3928
Process 4
L0: 15882
L1: 23624
L2: 60494
걸린 시간 : 3950
Process 7
L0: 16262
L1: 23985
L2: 59753
걸린 시간 : 3990
[Test 1] finished
done
$
```

다음은 건네주신 Test code를 실행한 결과입니다.

4개의 Process가 동작하며 각각 16 : 24 : 60의 비율로 옳게 동작하는 모습입니다.

비율 연산 방법 – $\text{time quantum} * \text{process num}$

Result – Scheduler Lock Test 1

```
int lock_test()
{
    parent = getpid();

    printf(1, "[Test 2] Lock Test 1 started\n");

    fork_children(1);
    if (getpid() != parent)
    {
        calculate();
        schedulerLock(123123);
        printf(1, "schedulerLock Not Kill!! Wrong....\n");
    }
    exit_children();

    fork_children(1);
    if (getpid() != parent)
    {
        schedulerLock(PASSWORD);
        calculate();
        schedulerUnlock(123123);
        printf(1, "schedulerUnlock Not Kill!! Wrong....\n");
    }
    exit_children();

    printf(1, "[Test 2] finished\n");
    printf(1, "done\n");
    return 0;
}
```

테스트 코드입니다.

```
Process 5
L0: 4162
L1: 6252
L2: 89586
결린 시간 : 973
kill (pid:5) (time quantum:0) (level:0) in schedulerLock for wrong password.
Process 6
L0: 3619
L1: 5744
L2: 90637
결린 시간 : 961
kill (pid:6) (time quantum:1) (level:0) in schedulerLock for wrong password.
[Test 2] finished
done
$ █
```


간단한 연산 이후 틀린 Password로 schedulerLock을 호출하니 Process가 종료된 모습입니다. schedulerUnlock도 마찬가지입니다.

Result – Scheduler Lock Test 2

```
int lock_test2()
{
    parent = getpid();
    printf(1, "[Test 2] Lock Test 2 started\n");

    fork_children(5);
    if (getpid() == parent)
    {
        schedulerLock(PASSWORD);
        printf(1, "Parent %d Lock!!\n", getpid());

        fork_children(1);
        if (getpid() != parent)
        {
            sleep(40);
            printf(1, "MLFQ 동작\n");
        }
        else
        {
            printf(1, "Parent %d Sleep!!\n", getpid());
            sleep(200);
            printf(1, "Parent %d Unlock!!\n", getpid());
            schedulerUnlock(PASSWORD);
        }
    }
    else
    {
        sleep(25);
        schedulerLock(PASSWORD);
        printf(1, "Child %d Lock!!\n", getpid());
        sleep(50);
        printf(1, "Child %d Unlock!!\n", getpid());
        schedulerUnlock(PASSWORD);
    }
    exit_children();
    printf(1, "[Test 2] finished\n");
    printf(1, "done\n");
    return 0;
}
```

테스트 코드입니다.

.

```
init: starting sh
$ mlfq_test
[Test 2] Lock Test 2 started
Parent 3 Lock!!
Parent 3 Sleep!!
MLFQ 동작
Parent 3 Unlock!!
Child 7 Lock!!
Child 7 Unlock!!
Child 4 Lock!!
Child 4 Unlock!!
Child 5 Lock!!
Child 5 Unlock!!
Child 6 Lock!!
Child 6 Unlock!!
Child 8 Lock!!
Child 8 Unlock!!
[Test 2] finished
done
$
```

순차적으로 Process가 Lock을 얻어 실행되는 모습입니다.

Result – Other System Call Test

```
int priority_test()
{
    int i;
    int count[MAX_LEVEL] = {0};

    parent = getpid();
    printf(1, "[Test 3] Priority Test 2 started\n");
    fork_children(4);

    if (getpid() != parent)
    {
        for (i = 0; i < NUM_LOOP; i++)
        {
            int x = getLevel();
            count[x]++;

            if (i%(getpid()*getpid()) == 0)
            {
                setPriority(getpid(), 0);
            }
            if (i%(getpid()*10) == 0)
            {
                yield();
            }
        }
        printf(1, "Success %d!!\n", getpid());
    }

    exit_children();
    printf(1, "[Test 3] finished\n");
    printf(1, "done\n");
    return 0;
}
```

```
init: starting sh
$ mlfq_test
[Test 3] Priority Test 2 started
Success 7!!
Success 6!!
Success 4!!
Success 5!!
[Test 3] finished
done
```

테스트 코드가 아무런 문제 없이 실행되는 모습입니다.

Result – Interrupt Process Trouble shooting

```
int main(int argc, char *argv[])
{

    asm volatile("int $128");
    asm volatile("int $129");

    exit();
}
```

테스트 코드입니다.

```
if(tf->trapno == T_SCHED_LOCK){
    cprintf("Lock 호출됨! \n");
    schedulerLock(2020003309);
    return;
}
if(tf->trapno == T_SCHED_UNLOCK){
    cprintf("Unlock 호출됨! \n");
    schedulerUnlock(2020003309);
    return;
}
```

trap 함수 내부 출력을 찍어냈습니다.

```
init: starting sh
$ mlfq_test
Lock 호출됨!
Unlock 호출됨!
$
```

결과가 잘 나오는 모습입니다.

Trouble Shooting

0. xv6 구조 파악

xv6가 어떤 구조인지 파악하는게 정말 난감했습니다. 동기들과 함께 모여서 4시간 정도 시간을 들여 겨우 기본 xv6의 TIMER 동작 과정과 Tick, Scheduler 등 어디를 고쳐야 하는가에 대한 결론이 나왔습니다.

1. Init Process Sleep & Not Working Shell

처음 MLFQ를 짜고, xv6를 부팅했을 때, Shell이 켜지지 않았습니다. 알고 봤더니, InitProc는 부팅 후 SLEEPING 상태로 들어가더군요. 제가 SLEEPING 상태에서 깨어났을 때 MLFQ에 Process를 넣어줘야 하는 케이스를 핸들링하지 않았더군요. 추가했더니 잘 됐습니다.

2. Global Tick

Global Tick인 ticks를 이용하여 boosting을 구현했더니 sleep 함수의 동작 과정이 굉장히 찝찝했습니다. ticks를 쓰는 다른 코드를 파악해보니 따로 tick을 선언해주는게 안전할 것 같았습니다. 그래서 boosting_ticks를 추가했습니다.

3. Lock Panic

Lock과 관련한 Panic이 굉장히 많이 일어났습니다. 이 과정에서 sched 함수와 scheduler 함수에서 ptable.lock의 동작과정과 context switch에 대해서 깊이 이해하게 되었습니다.

4. Priority Boosting

Test code 실행 중에는 비율이 잘 나와서 몰랐는데, 생각해보니 Process가 동작하지 않는 시간에도 boosting_tick은 올라가야 하며, 일정한 시간마다 Priority Boosting이 일어나야 했습니다. 초기에는 yield에만 Boosting을 구현해줘서 생긴 문제였습니다.

myproc이 없는 경우, myproc이 있지만 killed인 경우 등 여러 경우를 핸들링했습니다.

5. Args in case of Call to Interrupt

Scheduler Lock Interrupt를 호출해줄 때, Args로 Password를 어떻게 넘겨줄지 고민을 정말 많이 했어요. 한 5시간은 쓴 것 같습니다. 그런데 허무하게도 piazza에서 넘길 필요 없다. trap에서 Password를 넘겨주면 된다. 라고 하셔서 슬펐습니다.

6. Scheduler Lock 구현

Scheduler Lock 은 이렇게 동작해야 한다. 라는 명확한 디자인이 없다보니 어떻게 하는 게 맞는지에 대한 고민이 많았습니다. 제가 내린 결론은 Lock은 Unlock 을 호출하기 전까지 atomic하게 코드를 실행하기 위해 호출한다는 전제를 가지는 것이었습니다.

따라서 세마포어를 구현해야 했습니다. sleep 함수에서 p->chan 의 역할과 wakeup1에서 Process의 동작 과정에 대해 자세히 이해할 수 있었습니다. 구현에 성공한 것 같아요. 잘 동작합니다.

7. Process 개수 제한

현재 xv6는 할당 가능한 Process 개수가 64개로 제한되어 있습니다. 이것을 어떻게 해결할까 생각하다가, 현재 구현된 Ready Queue 를 제외하고 Unused Queue와 Sleeping Queue 등등을 구현해야 실현 가능하다고 봤습니다.

그런데 반쯤 구현하다보니 안타깝게도 제출일자까지 구현이 불가능하겠다는 결론이 나와서 었어버린 상태입니다.

kill의 경우, 모든 Process에서 주어진 pid를 찾지만, 실제로는 RUNNABLE, SLEEPING, RUNNING 인 프로세스만 탐색하면 됩니다. wait의 경우는 ZOMBIE 인 프로세스만 보면 됩니다. exit의 경우 SLEEPING, RUNNABLE 프로세스만 보면 됩니다. 이렇듯 시간만 더 있었다면 여러 Queue를 구현하고, 충분히 xv6 스케줄러에 적용이 가능했었다고 봅니다.

아쉽습니다.